



Revisão Acelerada de Programação - RAP RAP-2026, Marinha do Brasil

Centro de Instrução Almirante Alexandrino

Departamento de Aperfeiçoamento Avançado para Oficiais

Curso de Aperfeiçoamento Avançado em Guerra Acústica (C-ApA-Of-GA)

Professor: João Victor da Fonseca Pinto

Descrição do Documento: Lista de Exercícios 11

Código da Aula: 11

Lista de Exercícios 11 — Revisão Geral do Curso

Orientações e Instruções Gerais:

- Esta lista consolida os tópicos fundamentais ministrados ao longo de toda a disciplina, totalizando **100,0 pontos**.
- **Formato e Entrega:** As resoluções devem ser desenvolvidas e entregues em formato Jupyter Notebook (.ipynb) ou Google Colab na plataforma Google Classroom, na tarefa designada e impreterivelmente até o dia e horário determinados.
- Para questões de implementação de código, utilize os nomes de funções, assinaturas e **Type Hints** estritamente conforme especificado nos protótipos.
- Respeite as restrições específicas de cada tema (ex: não usar laços no Tema 1 ou no Tema 2).
- A pontuação de cada questão é atribuída de acordo com seu nível: **Fácil (1,0 ponto)**, **Médio (1,8 ponto)** e **Difícil (3,0 pontos)**.

Tema 1: Fundamentos, Variáveis, Operadores e Funções Básicas

Q1. [Fácil - Múltipla Escolha - 1,0 ponto] Considere a seguinte expressão aritmética executada em Python para calcular o valor total de uma compra com desconto:

```
total = 10 + 2 * 3 ** 2 - 8 // 4 + 7 % 3
```

Qual será o valor final atribuído à variável `total`?

- **Dica:** A ordem de precedência dos operadores aritméticos em Python é: 1º Parênteses `()`, 2º Exponenciação `**`, 3º Multiplicação `*`, Divisão `/`, Divisão Inteira `//` e Módulo `%` (avaliados da esquerda para a direita), 4º Adição `+` e Subtração `-` (avaliados da esquerda para a direita). O operador de divisão inteira `//` retorna apenas a parte inteira da divisão (ex: `8 // 4` resulta em `2`), enquanto o operador módulo `%` calcula o resto da divisão inteira (ex: `7 % 3` resulta em `1`).

(A) 25

(B) 27

- (C) 28
- (D) 30
- (E) 19

Q2. [Fácil - Múltipla Escolha - 1,0 ponto] Em Python, a conversão explícita de tipos (*type casting*) é muito utilizada ao processar dados de formulários web. Analise as quatro conversões abaixo:

```
v1 = int(7.89)
v2 = float("42.5")
v3 = bool("")
v4 = bool(0)
```

Quais são, respectivamente, os valores e tipos resultantes armazenados em `v1`, `v2`, `v3` e `v4`?

- (A) 7 (int), 42.5 (float), False (bool), False (bool)
- (B) 8 (int), 42.5 (float), True (bool), False (bool)
- (C) 7 (int), 42.5 (float), True (bool), True (bool)
- (D) 7.89 (float), 42.5 (float), False (bool), False (bool)
- (E) 8 (int), 42 (int), False (bool), False (bool)

Q3. [Fácil - Múltipla Escolha - 1,0 ponto] Sobre a diferença fundamental entre a instrução `print()` e o comando `return` dentro de uma função em Python, assinale a alternativa **CORRETA**:

- (A) `print()` armazena o valor em uma variável externa, enquanto `return` apenas exibe no terminal.
- (B) `print()` apenas exibe uma representação visual no console (efeito colateral), enquanto `return` devolve o dado para quem chamou a função e encerra sua execução.
- (C) Toda função que possui `print()` necessariamente retorna um valor numérico diferente de `None`.
- (D) O comando `return` é opcional e, caso omitido, a função sempre retornará a última mensagem impressa pelo `print()`.
- (E) `return` só pode ser utilizado para retornar números inteiros, enquanto `print()` funciona para qualquer tipo.

Q4. [Fácil - 1,0 ponto] Um viajante está planejando uma viagem internacional e precisa converter a previsão do tempo de graus Celsius para Fahrenheit. A fórmula de conversão é:

$$F = C \times 1.8 + 32$$

Escreva uma função chamada `celsius_para_fahrenheit` que receba a temperatura em Celsius e retorne o valor equivalente em Fahrenheit.

- **Protótipo:**

```
def celsius_para_fahrenheit(celsius: float) -> float:
```

- **Entrada:** 25.0

- **Saída:** 77.0

Q5. [Fácil - 1,0 ponto] Um supermercado está aplicando descontos percentuais no caixa. Escreva uma função chamada `aplicar_desconto` que receba o preço original de um produto (em reais) e a porcentagem de desconto (de 0 a 100) e retorne o valor final a ser pago pelo consumidor.

- **Protótipo:**

```
def aplicar_desconto(preco_original: float, porcentagem_desconto: float) -> float:
```

- **Entrada:** 150.0, 10.0

- **Saída:** 135.0

Q6. [Fácil - 1,0 ponto] Em uma clínica de nutrição, o Índice de Massa Corporal (IMC) é calculado pela fórmula:

$$\text{IMC} = \frac{\text{peso}}{\text{altura}^2}$$

onde o peso é dado em quilogramas e a altura em metros. Escreva uma função chamada `calcular_imc` que receba o peso e a altura de um paciente e retorne o valor do seu IMC.

- **Protótipo:**

```
def calcular_imc(peso_kg: float, altura_m: float) -> float:
```

- **Entrada:** 70.0, 1.75

- **Saída:** 22.857142857142858

Q7. [Fácil - 1,0 ponto] Um grupo de amigos foi a uma pizzeria e deseja dividir a conta em partes iguais. Ao valor consumido, deve ser acrescida a taxa de serviço de 10%. Escreva uma função chamada `rachar_conta` que receba o valor total consumido (em reais) e o número de pessoas na mesa, retornando o valor individual a ser pago por cada amigo.

- **Protótipo:**

```
def rachar_conta(valor_total: float, num_pessoas: int) -> float:
```

- **Entrada:** 200.0, 4

- **Saída:** 55.0

- Q8. [Médio - 1,8 ponto]** Em uma disciplina de programação, a média final de um estudante é calculada por uma média ponderada de três avaliações, com pesos 2, 3 e 5, respectivamente:

$$\text{Média} = \frac{N_1 \times 2 + N_2 \times 3 + N_3 \times 5}{2 + 3 + 5}$$

Escreva uma função chamada `media_ponderada_provas` que receba as três notas em ponto flutuante e retorne a média final ponderada calculada.

- **Protótipo:**

```
def media_ponderada_provas(n1: float, n2: float, n3: float) -> float:
```

- **Entrada:** 6.0, 7.0, 8.0

- **Saída:** 7.3

- Q9. [Médio - 1,8 ponto]** Um caixa eletrônico de autoatendimento precisa dispensar o menor número de cédulas possível para um determinado valor de saque inteiro (múltiplo de 10). O caixa dispõe de cédulas de R\$ 50, R\$ 20 e R\$ 10. Sem utilizar laços ou condicionais, escreva uma função chamada `sacar_cedulas` que receba o valor total do saque e retorne uma tupla com a quantidade de cédulas de 50, 20 e 10, nesta ordem.

- **Dica:** O operador de divisão inteira `//` obtém a quantidade inteira de vezes que um número cabe no outro (ex.: `180 // 50` resulta em 3), enquanto o operador módulo `%` obtém o resto da divisão (ex.: `180 % 50` resulta em 30).

- **Protótipo:**

```
from typing import Tuple
def sacar_cedulas(
    valor_reais: int
) -> Tuple[int, int, int]:
```

- **Entrada:** 180

- **Saída:** (3, 1, 1)

- Q10. [Difícil - 3,0 pontos]** Um investidor deseja calcular o montante acumulado em uma aplicação de renda fixa sob o regime de juros compostos. A fórmula matemática do montante é:

$$M = P \times (1 + i)^t$$

onde P é o capital inicial investido, i é a taxa de juros mensal expressa em decimal (ex: 1% = 0.01) e t é o número de meses de aplicação. Escreva uma função chamada `calcular_montante_juros_compostos` que receba o capital inicial, a taxa de juros mensal e o prazo em meses, e retorne o montante final acumulado.

- **Protótipo:**

```
def calcular_montante_juros_compostos(
    capital_inicial: float,
    taxa_mensal: float, meses: int
) -> float:
```

- Entrada: 1000.0, 0.01, 12
- Saída: 1126.8250301319697

Tema 2: Estruturas Condicionais e Lógica Booleana

Q11. [Fácil - Múltipla Escolha - 1,0 ponto] Em Python, a avaliação de expressões lógicas com os operadores `and` e `or` segue a regra do curto-circuito (*short-circuit evaluation*). Considere as variáveis:

```
a = False
b = True
c = True
```

Qual é o resultado da expressão lógica: `a and (b or c) or (not a and b)`?

- (A) False
- (B) True
- (C) None
- (D) Erro de sintaxe por excesso de parênteses
- (E) 1

Q12. [Fácil - Múltipla Escolha - 1,0 ponto] Considere o seguinte trecho de código em Python:

```
x = [1, 2, 3]
y = [1, 2, 3]
resultado1 = (x == y)
resultado2 = (x is y)
```

Quais são os valores booleanos de `resultado1` e `resultado2`?

- **Dica:** O operador `==` compara a **igualdade de valor/conteúdo** entre os objetos, enquanto o operador `is` compara a **identidade**, verificando se as variáveis apontam para a mesma referência/endereço na memória (`id(x) == id(y)`).
- (A) True e True
- (B) False e False
- (C) True e False
- (D) False e True
- (E) None e False

Q13. [Fácil - Múltipla Escolha - 1,0 ponto] Analise o seguinte bloco condicional em Python:

```
pontos = 75
if pontos >= 90:
    categoria = "Ouro"
elif pontos >= 70:
    categoria = "Prata"
elif pontos >= 50:
    categoria = "Bronze"
else:
    categoria = "Iniciante"
```

Qual valor estará armazenado na variável `categoria` ao final da execução?

- (A) "Ouro"
- (B) "Prata"
- (C) "Bronze"
- (D) "Iniciante"
- (E) "Prata" e em seguida "Bronze"

Q14. [Fácil - 1,0 ponto] Uma instituição de ensino adota o seguinte critério de avaliação acadêmica baseado na média final do discente:

- Média ≥ 7.0 : "APROVADO"
- $5.0 \leq \text{Média} < 7.0$: "RECUPERACAO"
- Média < 5.0 : "REPROVADO"

Escreva uma função chamada `status_aluno` que receba a média do aluno e retorne a string com seu status correspondente.

• **Protótipo:**

```
def status_aluno(nota_media: float) -> str:
```

- **Entrada:** 6.4 — **Saída:** "RECUPERACAO"
- **Entrada:** 8.5 — **Saída:** "APROVADO"

Q15. [Fácil - 1,0 ponto] Para se cadastrar como motorista em uma plataforma de transporte, a pessoa precisa ter no mínimo 18 anos de idade e possuir Carteira Nacional de Habilitação (CNH) válida. Escreva uma função chamada `pode_dirigir` que receba a idade (inteiro) e um booleano indicando se possui CNH, retornando `True` se a pessoa estiver apta a dirigir ou `False` caso contrário.

• **Protótipo:**

```
def pode_dirigir(idade: int, possui_cnh: bool) -> bool:
```

- **Entrada:** 19, `True` — **Saída:** `True`
- **Entrada:** 17, `True` — **Saída:** `False`

Q16. [Fácil - 1,0 ponto] Escreva uma função chamada `classificar_inteiro` que receba um número inteiro e o classifique em uma das quatro categorias: `"PAR_POSITIVO"`, `"PAR_NEGATIVO"`, `"IMPAR_POSITIVO"` ou `"IMPAR_NEGATIVO"`. Caso o número seja zero, a função deve retornar `"ZERO"`.

- **Protótipo:**

```
def classificar_inteiro(numero: int) -> str:
```

- **Entrada:** -4 — **Saída:** `"PAR_NEGATIVO"`
- **Entrada:** 7 — **Saída:** `"IMPAR_POSITIVO"`
- **Entrada:** 0 — **Saída:** `"ZERO"`

Q17. [Fácil - 1,0 ponto] No calendário gregoriano comercial, um ano é considerado bissexto se for divisível por 4, exceto se for divisível por 100, a menos que também seja divisível por 400. Escreva uma função chamada `eh_ano_bissexto` que receba o ano (número inteiro) e retorne `True` se for bissexto ou `False` caso contrário.

- **Protótipo:**

```
def eh_ano_bissexto(ano: int) -> bool:
```

- **Entrada:** 2024 — **Saída:** `True`
- **Entrada:** 1900 — **Saída:** `False`
- **Entrada:** 2000 — **Saída:** `True`

Q18. [Fácil - 1,0 ponto] Um e-commerce adota a seguinte política de entrega: o frete é totalmente gratuito (0.0) se o valor da compra for estritamente superior a R\$ 200,00 ou se o cliente for membro do programa VIP. Em qualquer outro caso, o valor do frete é fixo em R\$ 25,00. Escreva uma função chamada `calcular_frete` que implemente essa regra de negócio.

- **Protótipo:**

```
def calcular_frete(valor_compra: float, cliente_vip: bool) -> float:
```

- **Entrada:** 150.0, `True` — **Saída:** 0.0
- **Entrada:** 180.0, `False` — **Saída:** 25.0
- **Entrada:** 250.0, `False` — **Saída:** 0.0

Q19. [Fácil - 1,0 ponto] Para que três segmentos de reta a, b, c formem um triângulo válido, cada lado deve ser estritamente menor que a soma dos outros dois ($a < b + c$, $b < a + c$, $c < a + b$). Caso formem um triângulo, ele pode ser: `"EQUILATERO"` (3 lados iguais), `"ISOSCELES"` (2 lados iguais) ou `"ESCALENO"` (3 lados diferentes). Se não formarem triângulo, retorne `"INVALIDO"`. Escreva uma função chamada `tipo_triangulo`.

- **Protótipo:**

```
def tipo_triangulo(a: float, b: float, c: float) -> str:
```

- **Entrada:** 5.0, 5.0, 8.0 — **Saída:** "ISOSCELES"
- **Entrada:** 1.0, 2.0, 10.0 — **Saída:** "INVALIDO"

Q20. [Médio - 1,8 ponto] Considere uma tabela progressiva de Imposto de Renda mensal simplificada:

- Até R\$ 2.000,00: Isento (0.0)
- De R\$ 2.000,01 a R\$ 4.000,00: Alíquota de 10% sobre a parcela que exceder R\$ 2.000,00
- Acima de R\$ 4.000,00: R\$ 200,00 fixos mais 20% sobre a parcela que exceder R\$ 4.000,00

Escreva uma função chamada `calcular_ir_simplificado` que receba o salário bruto e retorne o valor do imposto devido.

- **Protótipo:**

```
def calcular_ir_simplificado(salario_bruto: float) -> float:
```
- **Entrada:** 3500.0 — **Saída:** 150.0
- **Entrada:** 5000.0 — **Saída:** 400.0

Q21. [Médio - 1,8 ponto] Um aplicativo de caronas cobra uma tarifa base de R\$ 5,00 mais R\$ 2,50 por quilômetro rodado. Caso a corrida ocorra em horário de pico, aplica-se um acréscimo de 30% sobre o valor total da corrida. Se estiver chovendo, é somada uma taxa adicional fixa de segurança de R\$ 4,00. Escreva uma função chamada `calcular_corrida` que receba a distância em km, se é horário de pico e se está chovendo, retornando o preço final.

- **Protótipo:**

```
def calcular_corrida(
    distancia_km: float,
    horario_pico: bool, esta_chovendo: bool
) -> float:
```
- **Entrada:** 10.0, True, True — **Saída:** 43.0
- **Entrada:** 10.0, False, False — **Saída:** 30.0

Q22. [Médio - 1,8 ponto] Escreva uma função chamada `validar_cadastro_usuario` que valide os dados de cadastro de um novo cliente em um sistema bancário digital. Os critérios obrigatórios são: 1. Idade deve ser maior ou igual a 18 e menor ou igual a 120 anos. 2. O email não pode ser vazio e deve conter o caractere "@" e pelo menos um caractere ".". 3. A renda mensal declarada não pode ser negativa. A função deve retornar uma tupla contendo um booleano (`True` se todos os critérios forem atendidos, `False` caso contrário) e uma mensagem explicativa ("CADASTRO_VALIDO" ou a causa da reprovação: "ERRO_IDADE", "ERRO_EMAIL" ou "ERRO_RENDA").

- **Protótipo:**

```
from typing import Tuple
def validar_cadastro_usuario(
    idade: int, email: str, renda: float
) -> Tuple[bool, str]:
```

- **Entrada:** 25, "joao@email.com", 3000.0 — **Saída:** (True, "CADASTRO_VALIDO")
- **Entrada:** 16, "ana@email.com", 1500.0 — **Saída:** (False, "ERRO_IDADE")

Q23. [Difícil - 3,0 pontos] Uma plataforma de streaming deseja recomendar o plano ideal para um usuário com base em suas preferências técnicas:

- **Plano Básico:** Suporta no máximo 1 tela simultânea, resolução padrão HD e não permite download offline.
- **Plano Padrão:** Suporta até 2 telas simultâneas, resolução Full HD (não 4K) e permite download offline.
- **Plano Premium:** Suporta até 4 telas simultâneas, resolução 4K Ultra HD e permite download offline.

Escreva uma função chamada `recomendar_plano_streaming` que receba o número de telas desejadas, se o usuário exige 4K e se exige download offline. A função deve retornar o nome do plano mais econômico que atenda plenamente aos requisitos ("PLANO_BASICO", "PLANO_PADRAO", "PLANO_PREMIUM") ou "PLANO_CUSTOMIZADO" se exceder 4 telas.

- **Protótipo:**

```
def recomendar_plano_streaming(
    num_telas: int, requer_4k: bool,
    requer_download: bool
) -> str:
```

- **Entrada:** 2, False, True — **Saída:** "PLANO_PADRAO"
- **Entrada:** 1, True, False — **Saída:** "PLANO_PREMIUM"
- **Entrada:** 5, True, True — **Saída:** "PLANO_CUSTOMIZADO"

Q24. [Difícil - 3,0 pontos] Um banco analisa pedidos de empréstimo pessoal com base em quatro variáveis: renda mensal líquida, valor da parcela pretendida, pontuação de score de crédito (0 a 1000) e existência de negativação no cadastro de inadimplentes. As regras de aprovação são:

1. O cliente não pode possuir negativação ativa (`tem_divida_ativa == False`).
2. O valor da parcela pretendida não pode ultrapassar 30% da renda mensal líquida.
3. Se o score for ≥ 700 , o empréstimo é aprovado com juros normais ("APROVADO_JUROS_BAIXOS").
4. Se o score estiver entre 500 e 699, o empréstimo é aprovado com taxa de risco ("APROVADO_JUROS_ALTOS").
5. Se o score for < 500 , o empréstimo é recusado por risco de crédito ("RECUSADO_SCORE").
6. Caso a parcela comprometa mais de 30% da renda, retorne "RECUSADO_MARGEM". Caso haja dívida ativa, retorne "RECUSADO_INADIMPLENCIA". A prioridade de rejeição é dívida ativa primeiro, depois margem e depois score. Escreva a função `analisar_credito_bancario`.

- **Protótipo:**

```
from typing import Tuple
def analisar_credito_bancario(
    renda_mensal: float, parcela: float,
    score: int, tem_divida_ativa: bool
) -> Tuple[bool, str]:
```

- **Entrada:** 5000.0, 1200.0, 750, False — **Saída:** (True, "APROVADO JUROS BAIXOS")
- **Entrada:** 4000.0, 1500.0, 800, False — **Saída:** (False, "RECUSADO MARGEM")

Tema 3: Estruturas de Repetição (for e while)

Q25. [Fácil - Múltipla Escolha - 1,0 ponto] Ao executar a instrução `list(range(10, 2, -3))` em Python, qual sequência de números inteiros será gerada?

- (A) [10, 7, 4]
- (B) [10, 7, 4, 1]
- (C) [10, 8, 6, 4, 2]
- (D) [10, 7, 4, 2]
- (E) [] (lista vazia)

Q26. [Fácil - Múltipla Escolha - 1,0 ponto] Considere o seguinte trecho de código em Python:

```
soma = 0
for i in range(1, 6):
    if i == 3:
        continue
    if i == 5:
        break
    soma += i
```

Qual será o valor final da variável `soma`?

- (A) 3
- (B) 7
- (C) 10
- (D) 15
- (E) 6

Q27. [Fácil - Múltipla Escolha - 1,0 ponto] Qual das seguintes situações causa obrigatoriamente um laço infinito (*infinite loop*) em Python?

- (A) Um laço `for i in range(1000):` sem o comando `break`.
- (B) Um laço `while condicao:` em que a variável presente na `condicao` nunca é modificada no corpo do laço e a condição permanece verdadeira.
- (C) Um laço que utiliza `continue` na primeira linha do corpo do `for`.
- (D) Percorrer uma tupla vazia com `for item in ():`.
- (E) Executar uma chamada recursiva que atinge seu caso base logo na primeira chamada.

Q28. [Fácil - Múltipla Escolha - 1,0 ponto] Quantas vezes a instrução `contador += 1` será executada no trecho abaixo?

```
contador = 0
for i in range(4):
    for j in range(3):
        contador += 1
```

- (A) 7 vezes
- (B) 12 vezes
- (C) 9 vezes
- (D) 16 vezes
- (E) 6 vezes

Q29. [Fácil - 1,0 ponto] Escreva uma função chamada `somar_pares_intervalo` que receba dois números inteiros positivos (`inicio` e `fim`, com $\text{inicio} \leq \text{fim}$) e, utilizando um laço de repetição `for`, calcule e retorne o somatório de todos os números pares pertencentes a esse intervalo fechado.

- **Protótipo:**

```
def somar_pares_intervalo(inicio: int, fim: int) -> int:
```

- **Entrada:** 1, 10 — **Saída:** 30 $*(2 + 4 + 6 + 8 + 10)*$
- **Entrada:** 4, 4 — **Saída:** 4

Q30. [Fácil - 1,0 ponto] Uma pessoa deseja comprar um equipamento eletrônico e decidiu guardar uma quantia fixa todo mês em um cofrinho (sem juros). Escreva uma função chamada `meses_para_meta_financeira` que receba o valor total da meta (em reais) e o valor do aporte mensal constante, utilizando um laço `while` para calcular e retornar a quantidade exata de meses necessários para acumular ou ultrapassar a meta.

- **Protótipo:**

```
def meses_para_meta_financeira(meta_reais: float, aporte_mensal: float) -> int:
```

- **Entrada:** 5000.0, 400.0 — **Saída:** 13

- **Entrada:** 1200.0, 300.0 — **Saída:** 4

Q31. [Fácil - 1,0 ponto] Escreva uma função chamada `gerar_tabuada` que receba um número inteiro N e, utilizando um laço `for`, retorne uma tupla com os 10 resultados da tabuada de multiplicação de N (de $N \times 1$ até $N \times 10$).

- **Protótipo:**

```
from typing import Tuple
def gerar_tabuada(numero: int) -> Tuple[int, ...]:
```

- **Entrada:** 7 — **Saída:** (7, 14, 21, 28, 35, 42, 49, 56, 63, 70)

Q32. [Fácil - 1,0 ponto] Escreva uma função chamada `contar_digitos_codigo` que receba uma string representando um código de rastreio de encomenda (por exemplo, "BR-2024-XP99-88") e, iterando caractere por caractere com um laço `for`, conte e retorne a quantidade de caracteres numéricos (dígitos de 0 a 9) presentes na string.

- **Protótipo:**

```
def contar_digitos_codigo(codigo: str) -> int:
```

- **Entrada:** "PED-2024-X99" — **Saída:** 6

- **Entrada:** "SEM_NUMEROS" — **Saída:** 0

Q33. [Fácil - 1,0 ponto] O fatorial de um número natural n (representado por $n!$) é o produto de todos os inteiros positivos menores ou iguais a n , com $0! = 1$. Escreva uma função chamada `calcular_fatorial` que receba um inteiro não negativo n e calcule seu fatorial utilizando um laço `while`.

- **Protótipo:**

```
def calcular_fatorial(n: int) -> int:
```

- **Entrada:** 5 — **Saída:** 120

- **Entrada:** 0 — **Saída:** 1

Q34. [Médio - 1,8 ponto] Um consumidor possui uma dívida de cartão de crédito que rende juros mensais fixos sobre o saldo restante. Todo mês, os juros são aplicados ao saldo devedor ($saldo = saldo \times (1 + taxa)$) e, logo após, o cliente realiza um pagamento fixo. Escreva uma função chamada `simular_pagamento_divida` que receba o saldo devedor inicial, a taxa de juros mensal em decimal e o valor do pagamento mensal fixo. A função deve retornar quantos meses serão necessários para quitar totalmente a dívida ($saldo \leq 0$). Caso o pagamento mensal seja menor ou igual aos juros do primeiro mês, a dívida nunca será paga; nesse caso, a função deve retornar -1.

- **Protótipo:**

```
def simular_pagamento_divida(
    saldo_devedor: float,
    taxa_juros_mensal: float,
    pagamento_mensal: float
) -> int:
```

- Entrada: 1000.0, 0.02, 200.0 — Saída: 6

- Entrada: 1000.0, 0.05, 50.0 — Saída: -1

Q35. [Médio - 1,8 ponto] Um número inteiro maior que 1 é primo se for divisível apenas por 1 e por ele mesmo. Escreva uma função chamada `eh_numero_primo` que receba um número inteiro e utilize um laço `for` para verificar se ele é primo, retornando `True` ou `False`. Números menores ou iguais a 1 não são primos.

- **Protótipo:**

```
def eh_numero_primo(numero: int) -> bool:
```

- Entrada: 17 — Saída: True

- Entrada: 21 — Saída: False

- Entrada: 1 — Saída: False

Q36. [Médio - 1,8 ponto] Escreva uma função chamada `extremos_gastos` que receba uma tupla não vazia de valores numéricos representando as despesas do mês e, utilizando apenas estruturas de repetição (sem usar as funções embutidas `min()` e `max()`), encontre e retorne uma tupla contendo o menor e o maior gasto, exatamente nesta ordem: `(menor, maior)`.

- **Protótipo:**

```
from typing import Tuple
def extremos_gastos(gastos: Tuple[float, ...]) -> Tuple[float, float]:
```

- Entrada: (120.5, 45.0, 890.0, 15.0, 320.0) — Saída: (15.0, 890.0)

Q37. [Difícil - 3,0 pontos] A sequência de Fibonacci é definida por: $F_0 = 0$, $F_1 = 1$ e $F_n = F_{n-1} + F_{n-2}$ para $n \geq 2$. Escreva uma função chamada `gerar_fibonacci` que receba a quantidade de termos desejada N ($N \geq 1$) e, utilizando um laço de repetição, gere e retorne uma tupla contendo os N primeiros termos da sequência de Fibonacci.

- **Protótipo:**

```
from typing import Tuple
def gerar_fibonacci(n_termos: int) -> Tuple[int, ...]:
```

- Entrada: 8 — Saída: (0, 1, 1, 2, 3, 5, 8, 13)

- Entrada: 2 — Saída: (0, 1)

Q38. [Difícil - 3,0 pontos] Em um jogo de adivinhação numérica, o computador escolhe um número inteiro entre 1 e um limite superior N . Um jogador inteligente utiliza a estratégia da Busca Binária para adivinhar o número no menor número de tentativas possível (sempre chutando o ponto médio do intervalo atual). Escreva uma função chamada `passos_busca_binaria` que receba o número secreto (`alvo`) e o `limite_superior`, e utilize um laço `while` para simular essa busca binária, retornando a quantidade de chutes/passos até acertar o número.

- **Protótipo:**

```
def passos_busca_binaria(alvo: int, limite_superior: int) -> int:
```

- **Entrada:** 73, 100 — **Saída:** 6

- **Entrada:** 50, 100 — **Saída:** 1

Tema 4: Strings, Fatiamento e Tuplas

Q39. [Fácil - Múltipla Escolha - 1,0 ponto] Dada a string `s = "PROGRAMACAO"`, qual será o resultado da operação de fatiamento `s[1:8:2]`?

(A) "RMAA"

(B) "RGAA"

(C) "RGA"

(D) "PGMC"

(E) "ROAM"

Q40. [Fácil - Múltipla Escolha - 1,0 ponto] Sobre a imutabilidade de strings e tuplas em Python, assinale a afirmação **CORRETA**:

(A) Podemos alterar um elemento de uma tupla diretamente com `t[0] = novo_valor` desde que o valor seja do mesmo tipo.

(B) O método `texto.replace("a", "b")` modifica a string `texto` no mesmo endereço de memória.

(C) Tanto strings quanto tuplas são objetos imutáveis; qualquer operação de concatenação ou substituição gera um novo objeto na memória.

(D) Tuplas que contêm apenas números inteiros são mutáveis, mas tuplas com strings são imutáveis.

(E) É possível deletar um caractere de uma string usando a instrução `del texto[0]`.

Q41. [Fácil - Múltipla Escolha - 1,0 ponto] Considere a string `frase = " banana,maca,uva "`. Ao executar o comando:

```
itens = frase.strip().split(",")
```

Qual será o valor armazenado na variável `itens`?

- (A) `"banana,maca,uva"`
- (B) `("banana", "maca", "uva")`
- (C) `["banana", "maca", "uva"]`
- (D) `[" banana", "maca", "uva "]`
- (E) `"banana maca uva"`

Q42. [Fácil - 1,0 ponto] Uma palavra ou frase é chamada de palíndromo se for lida da mesma maneira da esquerda para a direita e da direita para a esquerda, desconsiderando espaços em branco e diferenças entre maiúsculas e minúsculas. Escreva uma função chamada `eh_palindromo` que receba uma string e retorne `True` se for palíndromo ou `False` caso contrário.

- **Protótipo:**

```
def eh_palindromo(texto: str) -> bool:
```

- **Entrada:** `"Apos a sopa"` — **Saída:** `True`
- **Entrada:** `"Python"` — **Saída:** `False`
- **Entrada:** `"Socorram me subi no onibus em Marrocos"` — **Saída:** `True`

Q43. [Fácil - 1,0 ponto] Em trabalhos acadêmicos, a citação de autores no texto segue frequentemente o formato: `"SOBRENOME, P."` onde `SOBRENOME` é o último sobrenome em maiúsculas e `P.` é a primeira letra do primeiro nome seguida de ponto. Escreva uma função chamada `formatar_citacao_bibliografica` que receba o nome completo de um autor e retorne a citação formatada.

- **Protótipo:**

```
def formatar_citacao_bibliografica(nome_completo: str) -> str:
```

- **Entrada:** `"Carlos Eduardo Souza"` — **Saída:** `"SOUZA, C."`
- **Entrada:** `"Alan Turing"` — **Saída:** `"TURING, A."`

Q44. [Fácil - 1,0 ponto] Escreva uma função chamada `extrair_dominio_email` que receba uma string contendo um endereço de e-mail válido (ex: `"cliente.vip@lojaonline.com.br"`) e utilize os métodos de string para extrair e retornar apenas o domínio (a parte após o símbolo `"@"`).

- **Protótipo:**

```
def extrair_dominio_email(email: str) -> str:
```

- **Entrada:** `"contato@empresa.com.br"` — **Saída:** `"empresa.com.br"`
- **Entrada:** `"usuario@gmail.com"` — **Saída:** `"gmail.com"`

Q45. [Fácil - 1,0 ponto] Uma loja armazena as informações básicas de cada produto em uma tupla com quatro posições: `(codigo: int, nome: str, preco: float, categoria: str)`. Escreva uma função chamada `formatar_etiqueta_produto` que receba essa tupla, desempacote seus valores e retorne uma string formatada no padrão: `"[CATEGORIA] NOME - R$ PRECO (Cod: CODIGO)"`, onde o preço deve ser exibido com duas casas decimais.

- **Protótipo:**

```
from typing import Tuple
def formatar_etiqueta_produto(produto: Tuple[int, str, float, str]) -> str:
```

- **Entrada:** `(102, "Teclado Mecanico", 250.0, "Perifericos")`

- **Saída:** `"[Perifericos] Teclado Mecanico - R$ 250.00 (Cod: 102)"`

Q46. [Médio - 1,8 ponto] Um servidor web registra cada requisição em uma linha de log formatada por pares chave=valor separados por ponto e vírgula: `"IP=192.168.1.1; STATUS=200; BYTES=4096; PATH=/home"`. Escreva uma função chamada `parse_log_web` que receba essa string e faça a extração dos valores, retornando uma tupla tipada contendo `(ip: str, status: int, bytes: int, path: str)`.

- **Protótipo:**

```
from typing import Tuple
def parse_log_web(linha_log: str) -> Tuple[str, int, int, str]:
```

- **Entrada:**

```
"IP=192.168.1.1; STATUS=200; BYTES=4096; PATH=/home"
```

- **Saída:**

```
("192.168.1.1", 200, 4096, "/home")
```

Q47. [Médio - 1,8 ponto] A Cifra de César é uma técnica clássica de criptografia por substituição na qual cada letra de uma mensagem em maiúsculas é deslocada um número fixo de posições no alfabeto (considerando o alfabeto de 'A' a 'Z' de 26 letras de forma circular). Caracteres que não sejam letras maiúsculas (como espaços) devem ser mantidos inalterados. Escreva uma função chamada `cifrar_texto` que receba a mensagem e o deslocamento inteiro, retornando o texto cifrado.

- **Protótipo:**

```
def cifrar_texto(mensagem: str, deslocamento: int) -> str:
```

- **Entrada:** `"COMPUTADOR", 3` — **Saída:** `"FRPSXWDGRU"`

- **Entrada:** `"ZEBRA", 1` — **Saída:** `"AFCSB"`

Q48. [Difícil - 3,0 pontos] No algoritmo oficial do Cadastro de Pessoas Físicas (CPF), o primeiro dígito verificador (o 10º dígito) é calculado com base nos 9 primeiros dígitos: 1. Multiplica-se o 1º dígito por 10, o 2º por 9, o 3º por 8, e assim por diante até o 9º dígito multiplicado por 2.

2. Soma-se todos os 9 produtos parciais. 3. O resto da divisão dessa soma por 11 é calculado ($R = \text{soma} \pmod{11}$). 4. Se $R < 2$, o primeiro dígito verificador esperado é 0; caso contrário, é $11 - R$. Escreva uma função chamada `validar_primeiro_digito_cpf` que receba uma string de exatamente 11 dígitos numéricos e retorne `True` se o 10º dígito estiver correto ou `False` caso contrário.

- **Protótipo:**

```
def validar_primeiro_digito_cpf(cpf_str: str) -> bool:
```

- **Entrada:** `"12345678909"` — **Saída:** `True`

- **Entrada:** `"11144477705"` — **Saída:** `False`

Tema 5: Listas, Mutabilidade e Matrizes

Q49. [Fácil - Múltipla Escolha - 1,0 ponto] Considere duas listas: `a = [1, 2]` e `b = [3, 4]`. Qual é a diferença fundamental entre executar `a.append(b)` e `a.extend(b)`?

- (A) `a.append(b)` insere os elementos 3 e 4 individualmente no final de `a`, enquanto `a.extend(b)` cria uma cópia profunda.
- (B) `a.append(b)` insere a lista `b` inteira como um único elemento no final de `a` (resultando em `[1, 2, [3, 4]]`), enquanto `a.extend(b)` itera sobre `b` e anexa cada elemento individualmente (resultando em `[1, 2, 3, 4]`).
- (C) Não há nenhuma diferença prática; ambos os métodos resultam exatamente na lista `[1, 2, 3, 4]`.
- (D) `a.extend(b)` só funciona para listas de strings, enquanto `a.append(b)` funciona para qualquer tipo.
- (E) `a.append(b)` modifica `a` e retorna a nova lista, enquanto `a.extend(b)` não altera `a`.

Q50. [Fácil - Múltipla Escolha - 1,0 ponto] Analise o seguinte trecho de código em Python:

```
matriz_original = [[1, 2], [3, 4]]
matriz_copia = matriz_original.copy()
matriz_copia[0][0] = 99
```

Qual será o valor de `matriz_original[0][0]` após essa alteração?

- (A) 1, pois `.copy()` cria uma cópia totalmente independente de todos os níveis.
- (B) 99, pois `.copy()` realiza uma cópia rasa (*shallow copy*), compartilhando as referências das sublistas internas.
- (C) `None`, pois a lista original é limpa após a cópia.
- (D) Ocorre um erro de execução (`TypeError`) na linha 3.
- (E) 0.

Q51. [Fácil - Múltipla Escolha - 1,0 ponto] Qual das seguintes expressões em Python utiliza corretamente Compreensão de Listas (*List Comprehension*) para criar uma lista com o quadrado apenas dos números ímpares presentes na lista `numeros = [1, 2, 3, 4, 5, 6]`?

- (A) `[x ** 2 for x in numeros if x % 2 != 0]`
- (B) `[for x in numeros if x % 2 != 0: x ** 2]`
- (C) `[x ** 2 if x % 2 != 0 for x in numeros]`
- (D) `[x * 2 for x in numeros while x % 2 != 0]`
- (E) `list(x ** 2 for x in numeros and x % 2 != 0)`

Q52. [Fácil - 1,0 ponto] Um sistema de controle de vendas recebeu uma lista de preços com algumas entradas corrompidas contendo valores negativos ou iguais a zero. Escreva uma função chamada `limpar_precos_invalidos` que receba uma lista de preços em ponto flutuante e retorne uma nova lista contendo apenas os preços estritamente positivos (> 0.0).

- **Protótipo:**

```
from typing import List
def limpar_precos_invalidos(preco: List[float]) -> List[float]:
```

- **Entrada:** `[29.9, -5.0, 0.0, 45.5, 12.0]` — **Saída:** `[29.9, 45.5, 12.0]`

Q53. [Fácil - 1,0 ponto] Em uma competição esportiva, a nota de um competidor é calculada descartando apenas uma ocorrência da menor nota e uma ocorrência da maior nota atribuída pelos jurados, e calculando a média aritmética das notas restantes. Escreva uma função chamada `media_sem_extremos` que receba uma lista com pelo menos 3 notas e retorne a média aparada resultante.

- **Protótipo:**

```
from typing import List
def media_sem_extremos(notas: List[float]) -> float:
```

- **Entrada:** `[8.5, 9.0, 6.0, 10.0, 7.5]` — **Saída:** `8.333333333333334`

Q54. [Fácil - 1,0 ponto] Escreva uma função chamada `intercalar_listas` que receba duas listas de mesmo comprimento (`fila_a` e `fila_b`) e retorne uma nova lista contendo os elementos intercalados alternadamente (o primeiro de A, depois o primeiro de B, o segundo de A, o segundo de B, e assim por diante).

- **Protótipo:**

```
from typing import List
def intercalar_listas(fila_a: List[str], fila_b: List[str]) -> List[str]:
```

- **Entrada:** `["Ana", "Carlos"], ["Bia", "Daniel"]`
- **Saída:** `["Ana", "Bia", "Carlos", "Daniel"]`

Q55. [Fácil - 1,0 ponto] Escreva uma função chamada `trocar_pares_adjacentes` que receba uma lista de inteiros com quantidade par de elementos e retorne uma nova lista na qual as posições dos pares vizinhos estejam invertidas: o elemento no índice 0 troca com o índice 1, o elemento no índice 2 troca com o índice 3, e assim por diante.

- **Protótipo:**

```
from typing import List
def trocar_pares_adjacentes(lista_itens: List[int]) -> List[int]:
```

- **Entrada:** `[10, 20, 30, 40]` — **Saída:** `[20, 10, 40, 30]`

- **Entrada:** `[1, 2, 3, 4, 5, 6]` — **Saída:** `[2, 1, 4, 3, 6, 5]`

Q56. [Médio - 1,8 ponto] Uma imagem digital monocromática é representada por uma matriz bidimensional de dimensões $M \times N$ (uma lista de M listas, cada uma com N inteiros correspondentes aos tons de cinza). A rotação e transposição dessa imagem exige converter as linhas em colunas. Escreva uma função chamada `transpor_imagem` que receba essa matriz e retorne a matriz transposta de dimensão $N \times M$.

- **Protótipo:**

```
from typing import List
def transpor_imagem(matriz_pixels: List[List[int]]) -> List[List[int]]:
```

- **Entrada:** `[[1, 2, 3], [4, 5, 6]]` — **Saída:** `[[1, 4], [2, 5], [3, 6]]`

Q57. [Médio - 1,8 ponto] Uma rede de lojas armazena em uma matriz as quantidades de produtos vendidos em cada filial (cada linha representa uma filial e cada coluna um tipo de produto). Um vetor unidimensional armazena o preço unitário de cada produto. Escreva uma função chamada `faturamento_por_departamento` que realize a multiplicação da matriz de vendas pelo vetor de preços, retornando uma lista com o faturamento total arrecadado por cada filial.

- **Protótipo:**

```
from typing import List
def faturamento_por_departamento(
    matriz_vendas: List[List[float]],
    vetor_precos: List[float]
) -> List[float]:
```

- **Entrada:** `[[2.0, 1.0], [0.0, 3.0]], [10.0, 20.0]` — **Saída:** `[40.0, 60.0]`

Q58. [Difícil - 3,0 pontos] No mercado financeiro, a Média Móvel Simples é muito utilizada para suavizar oscilações diárias de preços. Dada uma lista de preços históricos de uma ação e um tamanho de janela K ($K \geq 1$), a média móvel no dia i é a média aritmética dos K valores compreendidos entre o dia $i - K + 1$ e o dia i . Escreva uma função chamada `calcular_media_movel` que receba a lista de preços e a janela K , e retorne uma lista contendo as médias móveis calculadas a partir do momento em que a primeira janela completa estiver disponível.

- **Protótipo:**

```
from typing import List
def calcular_media_movel(precos_diarios: List[float], tamanho_janela: int) -> List[float]:
```

- **Entrada:** [100.0, 102.0, 104.0, 106.0, 108.0], 3 — **Saída:** [102.0, 104.0, 106.0]

Tema 6: Dicionários e Conjuntos (dict e set)

Q59. [Fácil - Múltipla Escolha - 1,0 ponto] Dado o dicionário `precos = {"arroz": 5.50, "feijao": 8.00}`, qual é a forma segura de consultar o preço do item "batata" sem provocar uma exceção de erro do tipo `KeyError` caso ele não esteja cadastrado?

- (A) `precos["batata"]`
- (B) `precos.get("batata", 0.0)`
- (C) `precos.pop("batata")`
- (D) `precos.find("batata")`
- (E) `precos.search("batata")`

Q60. [Fácil - 1,0 ponto] Escreva uma função chamada `contar_frequencia_palavras` que receba uma string contendo um texto em minúsculas com palavras separadas por espaços e retorne um dicionário no qual cada chave é uma palavra e o valor correspondente é a quantidade de vezes que ela apareceu no texto.

- **Protótipo:**

```
from typing import Dict
def contar_frequencia_palavras(texto: str) -> Dict[str, int]:
```

- **Entrada:** "otimo produto produto excelente otimo produto"
- **Saída:** {"otimo": 2, "produto": 3, "excelente": 1}

Q61. [Fácil - 1,0 ponto] Escreva uma função chamada `inverter_dicionario` que receba um dicionário cujas chaves e valores sejam strings (assumindo que todos os valores são distintos) e retorne um novo dicionário no qual as chaves originais tornam-se valores e os valores tornam-se as novas chaves.

- **Protótipo:**

```
from typing import Dict
def inverter_dicionario(dicionario_original: Dict[str, str]) -> Dict[str, str]:
```

- **Entrada:** {"BR": "Brasil", "FR": "Franca"}

- Saída: {"Brasil": "BR", "Franca": "FR"}

Q62. [Fácil - 1,0 ponto] Um catálogo de supermercado possui itens cadastrados em uma lista de tuplas no formato (nome_produto, categoria). Escreva uma função chamada `agrupar_por_categoria` que receba essa lista e retorne um dicionário no qual cada chave é o nome de uma categoria e o valor é a lista de produtos pertencentes àquela categoria.

- Protótipo:

```
from typing import List, Tuple, Dict
def agrupar_por_categoria(
    itens: List[Tuple[str, str]]
) -> Dict[str, List[str]]:
```

- Entrada:

```
[("Arroz", "Alimentos"), ("Sabonete", "Higiene"), ("Feijao", "Alimentos")]
```

- Saída:

```
{"Alimentos": ["Arroz", "Feijao"], "Higiene": ["Sabonete"]}
```

Q63. [Médio - 1,8 ponto] Duas filiais de uma rede de lojas precisam consolidar seus estoques ao final do expediente. Cada estoque é representado por um dicionário no formato {"produto": quantidade}. Escreva uma função chamada `consolidar_estoques` que receba os dois dicionários e retorne um novo dicionário consolidado contendo a soma das quantidades para produtos que existirem em ambas as lojas, e a quantidade original para os produtos exclusivos de uma única loja.

- Protótipo:

```
from typing import Dict
def consolidar_estoques(
    estoque_loja1: Dict[str, int],
    estoque_loja2: Dict[str, int]
) -> Dict[str, int]:
```

- Entrada: {"Camisa": 10, "Calca": 5}, {"Camisa": 8, "Tenis": 3}

- Saída: {"Camisa": 18, "Calca": 5, "Tenis": 3}

Tema 7: Programação Orientada a Objetos e Manipulação de Arquivos

Q64. [Fácil - Múltipla Escolha - 1,0 ponto] Em Programação Orientada a Objetos em Python, qual é o papel do primeiro parâmetro `self` na definição dos métodos de instância dentro de uma classe?

- (A) `self` é uma palavra-chave reservada que faz referência direta à classe mãe (superclasse).

- (B) `self` representa a instância específica do objeto que está invocando o método, permitindo acessar e manipular seus atributos e outros métodos.
- (C) `self` serve exclusivamente para definir variáveis estáticas compartilhadas entre todas as instâncias da classe.
- (D) O uso de `self` é opcional e pode ser substituído pela palavra-chave `global` para criar atributos públicos.
- (E) `self` é o nome do método construtor padrão executado para alocar memória ao objeto.

Q65. [Fácil - Múltipla Escolha - 1,0 ponto] Por que a comunidade de desenvolvedores Python recomenda fortemente o uso do gerenciador de contexto `with open(...)` ao invés de usar `f = open(...)` seguido de `f.close()`?

- (A) Porque a instrução `with` lê o arquivo de forma assíncrona mais rápida.
- (B) Porque o gerenciador de contexto `with` garante o fechamento automático e a liberação dos recursos do arquivo no sistema operacional, mesmo se ocorrer uma exceção ou erro durante o processamento.
- (C) Porque a função `open()` tradicional é considerada obsoleta no Python 3.
- (D) Porque arquivos abertos com `with` não ocupam memória RAM.
- (E) Porque o `with` converte automaticamente o arquivo de texto para formato binário criptografado.

Q66. [Fácil - 1,0 ponto] Implemente uma classe chamada `Livro` que represente um exemplar de leitura de uma biblioteca com os seguintes requisitos: 1. Construtor `__init__(self, titulo: str, autor: str, total_paginas: int)` que inicialize os atributos e crie o atributo `paginas_lidas: int` iniciado em 0. 2. Método `ler_paginas(self, quantidade: int) -> int` que adicione a quantidade lida a `paginas_lidas` (sem ultrapassar `total_paginas`) e retorne quantas páginas ainda faltam para concluir o livro. 3. Método `print(self) -> None` que exiba na tela a mensagem formatada: `"'TITULO' por AUTOR (Lidas: X/TOTAL)"`.

Exemplo de Uso:

```
livro = Livro("1984", "George Orwell", 300)
faltam = livro.ler_paginas(50) # retorna 250
livro.print() # exibe: '1984' por George Orwell (Lidas: 50/300)
```

Q67. [Fácil - 1,0 ponto] Escreva uma função chamada `contar_erros_log` que receba o caminho de um arquivo de texto `.txt` de log de uma aplicação web. Utilizando o gerenciador de contexto `with`, a função deve abrir o arquivo no modo leitura (`'r'`), percorrer todas as linhas e contar quantas contêm a palavra `"ERROR"` (em maiúsculas), retornando essa contagem como um número inteiro.

• **Protótipo:**

```
def contar_erros_log(caminho_arquivo: str) -> int:
```

- **Entrada:** "servidor.log" (arquivo contendo 5 ocorrências de "ERROR")
- **Saída:** 5

Q68. [Fácil - 1,0 ponto] Escreva uma função chamada `salvar_produtos_csv` que receba o caminho de um arquivo de saída e uma lista de tuplas contendo `(nome_produto: str, preco: float)`. A função deve utilizar a cláusula `with` no modo de escrita (`'w'`) para gravar um arquivo CSV com cabeçalho `"produto,preco"` e cada item em uma nova linha com o preço formatado em duas casas decimais, retornando `True` após finalizar a gravação.

- **Protótipo:**

```
from typing import List, Tuple
def salvar_produtos_csv(caminho_arquivo: str, produtos: List[Tuple[str, float]]) -> bool:
```

- **Entrada:** "precos.csv", `[("Mouse", 80.0), ("Teclado", 150.0)]`
- **Saída:** `True` (e o arquivo "precos.csv" salvo no disco)

Q69. [Fácil - 1,0 ponto] Implemente uma classe chamada `ContaBancaria` que gerencie o saldo de um cliente com os seguintes métodos e atributos: 1. Construtor `__init__(self, titular: str, saldo_inicial: float = 0.0)` que inicialize o titular e o saldo da conta. 2. Método `depositar(self, valor: float) -> float` que adicione o valor ao saldo (se o valor for positivo) e retorne o novo saldo atualizado. 3. Método `sacar(self, valor: float) -> bool` que debite o valor do saldo se houver saldo suficiente ($saldo \geq valor$ e $valor > 0$) retornando `True`. Caso contrário, não altera o saldo e retorna `False`. 4. Método `print(self) -> None` que exiba na tela a mensagem formatada: `"Conta de TITULAR - Saldo: R$ X.XX"`.

Exemplo de Uso:

```
conta = ContaBancaria("Maria", 100.0)
conta.depositar(50.0) # Saldo: 150.0
sucesso = conta.sacar(200.0) # Retorna False (saldo insuficiente)
conta.print() # Exibe: Conta de Maria - Saldo: R$ 150.00
```

Q70. [Médio - 1,8 ponto] Modele uma hierarquia de classes para uma concessionária de veículos: 1. Superclasse `Veiculo`: * Construtor `__init__(self, marca: str, modelo: str, preco: float)`. * Método `exibir_ficha(self) -> str` que retorne `"MARCA MODELO - R$ PRECO"`. 2. Subclasse `CarroEletrico` (que herda de `Veiculo`): * Construtor `__init__(self, marca: str, modelo: str, preco: float, capacidade_kwh: float)` utilizando `super()` para os atributos herdados. * Método `estimar_autonomia(self, consumo_kwh_por_km: float) -> float` que retorne o alcance estimado em km ($\frac{capacidade_kwh}{consumo_kwh_por_km}$).

Exemplo de Uso:

```
carro = CarroEletrico(
    "Tesla", "Model 3", 280000.0, 75.0
)
```

```
# Ficha: Tesla Model 3 - R$ 280000.00
print(carro.exibir_ficha())
# Autonomia estimada: 500.0 km
autonomia = carro.estimar_autonomia(0.15)
```

Q71. [Médio - 1,8 ponto] Uma plataforma de e-commerce mantém seus pedidos salvos em um arquivo CSV formatado com cabeçalho:

```
id,cliente,valor
1,Maria Silva,120.50
2,Joao Souza,45.00
```

Escreva uma função chamada `carregar_pedidos_csv` que leia o arquivo informado via parâmetro usando `with` e retorne uma lista de dicionários tipados no formato `[{"id": int, "cliente": str, "valor": float}]`. Linhas em branco devem ser ignoradas.

- **Protótipo:**

```
from typing import List, Dict, Union
def carregar_pedidos_csv(
    caminho_csv: str
) -> List[Dict[str, Union[int, str, float]]]:
```

- **Entrada:** "pedidos.csv"

- **Saída:**

```
[{"id": 1, "cliente": "Maria Silva", "valor": 120.50}, {"id": 2, "cliente":
"Joao Souza", "valor": 45.00}]
```

Q72. [Difícil - 3,0 pontos] Desenvolva um sistema completo de controle de acervo para uma biblioteca comunitária criando a classe `Biblioteca`:

- Construtor `__init__(self, nome_biblioteca: str)`: inicializa o nome e o catálogo interno `self.catalogo: Dict[int, Dict[str, Union[str, int]]]`.
- Método `cadastrar_livro(self, livro_id: int, titulo: str, quantidade: int) -> None`: adiciona ou atualiza os dados do livro no catálogo.
- Método `emprestar_livro(self, livro_id: int) -> bool`: se o livro existir e possuir quantidade disponível (> 0), decrementa em 1 unidade e retorna `True`. Caso contrário, retorna `False`.
- Método `exportar_inventario(self, caminho_txt: str) -> bool`: grava um arquivo de texto com a lista de todos os livros no formato `"[ID] TITULO - QTD exemplares"` e encerra com a linha final `"Total de titulos: N"`, retornando `True`.

Exemplo de Uso:


```
bib = Biblioteca("Biblioteca Central")
bib.cadastrar_livro(101, "Dom Casmurro", 3)
# Empresta: qtd vira 2, retorna True
bib.emprestar_livro(101)
# Grava inventario no disco
bib.exportar_inventario("inventario.txt")
```